

Cluster of Clusters

Felicia Tao, Blake Muxlow, Tahseen Hussain, Afraz Hameed
CSE 145/237D, 11.06.2026

1. Team Updates & Addressing Feedback

Recently, our team was reduced from five members to four, as Andre dropped the course. We have restructured our task delegation and future milestones accordingly in response to this change.

In our initial project specification, we received feedback regarding unclear team roles, lack of stated technical expertise, undefined milestone priorities, and missing task owners on our Gantt chart. We have addressed these below and made sure that any future tasks are clearly labeled with the responsible party for accountability.

1.1. Team Roles & Task Owners

For this second half of the quarter, our team is split into 2: Kind and KWOK. Blake and Tahseen are working on KWOK together, with Blake in charge of deployment and automation and Tahseen in charge of experiments, metric collection, and analysis. A similar format is followed for Kind, with Felicia working on deployment and automation and Afraz doing experiments, metric collection, and analysis. We are all in charge of justifying, validating the simple rule we came up with, making changes and updates to the rule as needed.

1.2. Team Technical Expertise

- Felicia: Docker and Kubernetes Containerization, scripting and automation pipelines

- Blake: Kubernetes, Grafana, Linux System Administration, Docker, scripting, YAML, CI/CD, KWOK
- Tahseen: Observability interrogation, workload metric collection and benchmarking, Karmada/Thanos Federation
- Afraz: Linux, Cluster Management, CLI, Data Visualization, Data Modeling and Analysis

1.3. Milestone Priorities

We have clarified which milestones are our priorities moving forward.

- High Priority: Kind + KWOK integration and experiments, baseline hardware testing, comparisons/analysis between Kind/KWOK simulations and reality
- Medium Priority: Detailed hardware testing (stress and availability testing)
- Low Priority: Implementing CI/CD pipelines via ArgoCD, homogeneous vs heterogeneous mixture of hardware testing

2. MVP & Milestone Completion

Our MVP was defined as a locally simulated cluster of cluster environment running on Kind and managed by Karmada, capable of deploying workloads across multiple sub clusters. The system should support basic cross cluster communication, resource monitoring via Prometheus & Grafana, and a workload process to simulate user activity.

We have successfully achieved this result, as shown in our oral project update, through the completion of the promised deliverables 1 and 2, along with some parts of deliverable 3. To ensure clear group management and accountability, the main milestones were divided and accomplished by individual team members as follows:

2.1. Deliverable 1: Project Specification and Environment Setup

2.1.1. Milestone: Complete Gantt Chart

Felicia created the [Gantt chart](#) organization and tasks, along with mapping out the expected timeline over the quarter.

2.1.2. Milestone: Complete Project Specification

The entire group worked together on completing the project specification. More specifically Tahseen and Afraz worked together on the project charter section, Blake did the group management and project development sections, and Felicia did the project milestones and schedule section.

2.1.3. Milestone: Learn Technical Stack

The entire team learned the base technical stack, including Kubernetes, Kind, Karmada, Prometheus, and Grafana.

2.1.4. Milestone: Install Required Dependencies

Everyone in the team installed the required dependences and made sure they worked to ensure that each person had a proper environment to work in.

2.2. Deliverable 2: Simulated Cluster of Cluster Environment and Initial Analysis

2.2.1. Milestone: Use Docker, Kubernetes, and Kind to create local worker environments

Blake and Felicia were responsible for first setting up some basic commands needed to get Kind and Kubernetes running to create a simulated cluster of clusters environment. The scripts with these commands can be found [here](#).

2.2.2. Milestone: Use Karmada to deploy a control plane and manage the local clusters

Tahseen was responsible for figuring out how to have Karmada manage all the clusters and be capable of federating the workload. This included figuring out how the clusters communicated with each other. Below is an instance of a Karmada managed multicluster deployment environment integrating Prometheus, Grafana, and federated control plane services

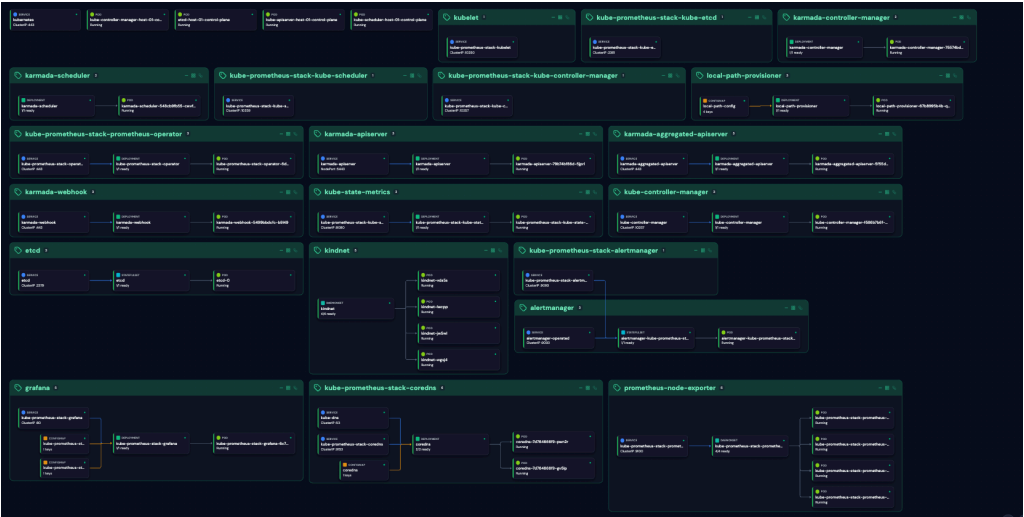


Figure 1: Karmada Network

) A major portion of the work involved understanding and configuring how clusters communicate with one another, include cluster registration, API access

between clusters, and networking issues. One big challenge in this process were issues with IP range conflicts between the simulated Docker containers KIND uses to emulate nodes and access outside of them. That was handled via dynamic IP handling of the spun on container-nodes to capture the emulated machines IP addresses and using those for access. This challenge seems unique to Windows & MacOS machines as Docker is run in a Linux VM managed by the Docker Engine.

2.2.3. Milestone: Configure Admiralty/Karmada to simulate DSMLP federation

This was a milestone we originally had but ended up removing from our project scope as it would introduce many external dependencies with DSMLP. Having to establish all the necessary network connections and access with university infrastructure would not have been feasible. In addition, the exact reason for this desired feature was never fully communicated as it was just thought of as something that could be nice if possible.

2.2.4. Milestone: Use Prometheus and Grafana to collect and display resource usage statistics

Felicia was responsible for setting up the base commands for installing Prometheus on all the clusters and using Grafana to gather all the metrics and view them. Tahseen then took this one step further by getting Prometheus and Grafana connected to Karmada so that we could get data from the control cluster too, not just worker ones. The scripts for these can be found [here](#) and [here](#). Additionally, Prometheus and Grafana are used with KWOK to display API call latency and control plane metrics.

Tahseen also researched further into large scale observation strategies and investigated Prometheus federation and Thanos as solutions for aggregating metrics across clusters. In large multicluster environments, a centralized metrics layer becomes more necessary because individual Prometheus instances do not scale efficiently for long term storage or high query loads. Prometheus federation addresses this by allowing metrics to be selectively aggregated from multiple instances in a lightweight manner, enabling analysis without overloading any single node. Thanos builds on this further by providing long term retention via object storage, and global query views across all clusters, features that become more essential as infrastructure scales. To simulate and validate monitoring at scale, KWOK was first set up to create lightweight fake clusters without the overhead of real nodes. Thanos was then deployed on top to aggregate metrics across these simulated clusters, demonstrating how the observability stack holds up under large multicluster conditions.

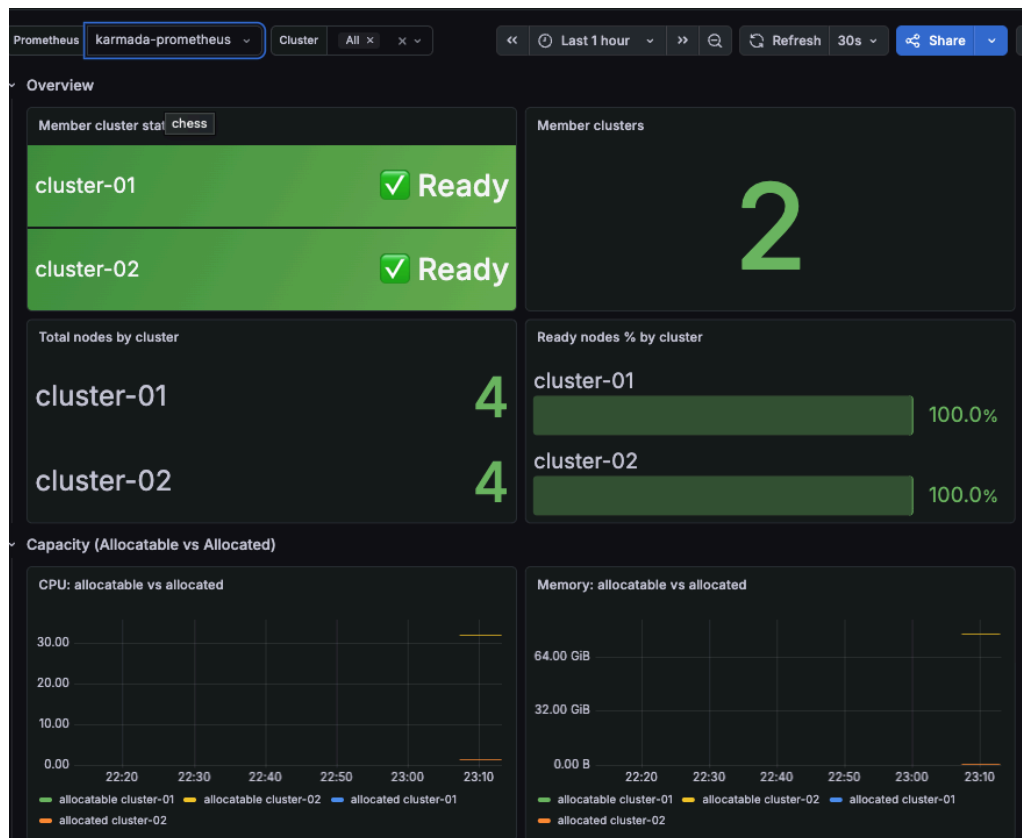


Figure 2: Karmada-API dashboard displayed in Grafana

2.2.5. Milestone: Create deployment workload script to simulate user activity

Afraz was responsible for a script to create simulated workloads that would represent user activity to run on the cluster topologies. He was also responsible for stress testing clusters and figuring out the point at which clusters would bottleneck and slow down.

Afraz created a file for simulating workloads to represent activity including memory, CPU, and mixed tests and allowing customization of load testing. [File in GitHub](#)

Developed a manual for it giving a basic guide and overview in the same folder called kind-stress-guide.

2.2.6. Milestone: Research and test baseline topologies and document performance

Felicia did the experiment suite of the 9 different topology configuration and gathered the metrics to do analysis with. Metrics/graphs tracked can be found [here](#).

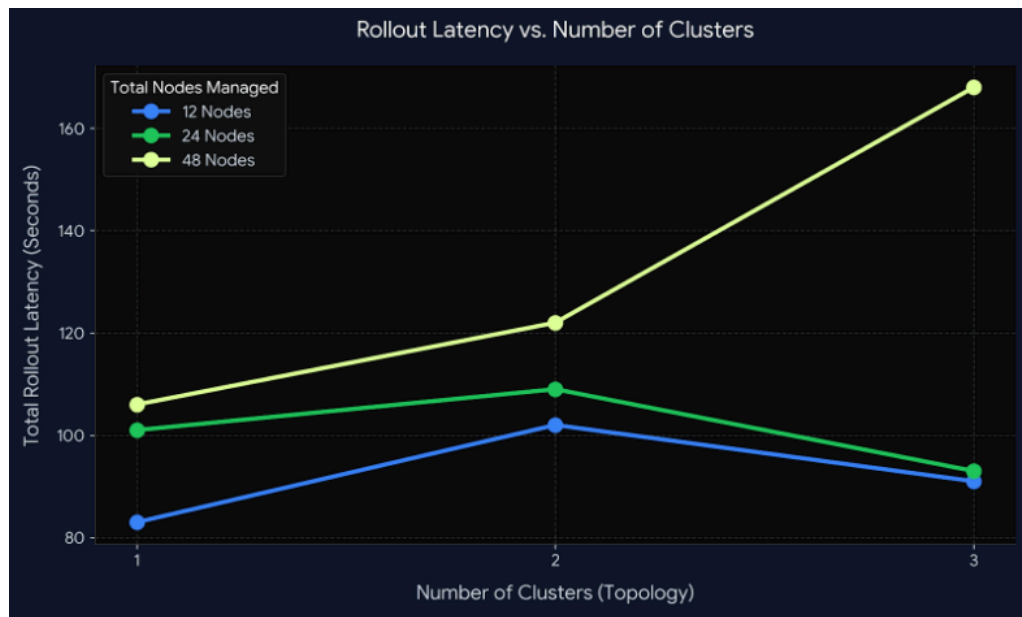


Figure 3: Latency Overhead of Different Topologies

2.2.7. Milestone: Prepare Oral Project Update

Everyone contributed to the creation of the [oral project update slides](#), the analysis of data, and the team as a whole practiced our presentation together.

2.3. Deliverable 3: Automation Pipeline and Deeper Analysis (Partially Complete)

2.3.1. Milestone: Use Ansible/Terraform to write automated pipelines from topology creation to data collection

Felicia was responsible for creating most of the automated pipelines for spinning up cluster topologies and running them for tests as a whole. Tahseen's work contributed to the Karmada portion and Afraz's work contributed the workload deployment portion. The resulting scripts can be found [here](#) and [here](#).

2.3.2. Milestone: Find the maximum number of clusters that can be handled by Kubernetes and Karmada through load testing

Felicia did some maximum limits testing to find the total number of nodes that can be in one cluster (all these nodes would be managed by the Kubernetes control plane) and the maximum number of 1-node clusters (all clusters managed by Karmada). Scripts for these experiments can be found [here](#).

3. Milestone Revisions / Additions

While working on deliverable 2, we realized a critical limitation in our software stack that required us to pivot directions. Kind was expected to allow us to simulate topologies of 1000+ nodes, but in reality, it used real resources which made us run into issues really soon. Because of this, Blake pivoted and worked on 2 new milestones: (1) finding a more suitable software to use and (2) spinning up clusters with that new software, see what limitations it has, and decide if we should move forward with it. Blake ended up finding and experimenting with KWOK, and as a team, we decided that it was worth shifting our milestones to incorporate it so that we are able to further scale our topology experiments. So, we have pivoted our future milestones to include experiments both in Kind and KWOK to both compare them against each other, use them together, and scale our experiments as much as possible.

4. Future Milestones & Timeline

4.1. Deliverable 3: Automation Pipeline and Deeper Analysis (Continued) - Week 7/8

4.1.1. Milestone: Review feedback from update and adjust schedule as needed

Integrate feedback from the oral update presentation to refine our project goals.
Role: All *Priority:* High

4.1.2. Milestone: Improve data collection and visualization dashboard

Update Grafana dashboards to only display information we are interested in collecting and start looking into how to collect hardware data, if even possible.
Role: Tahseen(KWOK) and Afraz(Kind) *Priority:* High

4.1.3. Milestone: Run more experiments and analyze data to create a set of rules for when clusters should be split

Further experiment with the Kind and KWOK setup to find the exact points where parallelization speed overtakes network scheduling overhead on larger scales and varying workloads. Also experiment with combining Kind and KWOK to have real and fake nodes running together so metrics gathered are more reliable.
Role: Blake(KWOK) and Felicia(Kind) *Priority:* High

4.2. Deliverable 4: Hardware Setup - Week 8

4.2.1. Milestone: Configure OS and local network on Pixel phones

Prepare the Pixel phones by setting up the base operating system and ensuring reliable local networking so they can communicate seamlessly with each other.

Role: Felicia *Priority:* High

4.2.2. Milestone: Deploy lightweight version of Kubernetes

Install a lightweight Kubernetes distribution (such as K3s or MicroK8s) onto the physical devices for the management of the containerized clusters and its nodes.

Role: Blake *Priority:* High

4.2.3. Milestone: Connect all nodes to Karmada control plane

Link the physical Pixel worker clusters back to the central Karmada control plane cluster running on a host machine, establishing the physical “Cluster of Clusters.”

Role: Tahseen *Priority:* High

4.2.4. Milestone: Apply Admiralty/Karmada access policies

Configure the necessary propagation policies in Karmada to ensure workloads are properly and securely distributed across the physical hardware Pixel phones.

Role: Tahseen *Priority:* High

4.2.5. Milestone: Modify data collection to account for hardware constraints

Adjust the observability stack to accurately measure the limitations of the Pixel phones, such as thermal throttling, memory limits, and Wi-Fi network latency.

Role: Afraz *Priority:* High

4.3. Deliverable 5: Hardware Experiments and Testing - Week 9

4.3.1. Milestone: Run baseline testing to ensure hardware works as expected

Deploy a simple, low-resource workload to the Pixel clusters to verify that the control plane can successfully schedule and monitor pods on physical devices.

Role: All *Priority:* High

4.3.2. Milestone: Execute best topologies found so far in simulation

Replicate our most efficient 1, 2, and 3-cluster setups on the physical hardware to validate whether our simulation-based analysis and rules hold true in reality.

Role: All *Priority:* High

4.3.3. Milestone: Execute hardware stress testing and high availability testing

Push the physical devices to their limits (CPU/RAM saturation) to observe how Karmada handles physical node failures, database timeouts, and network drops.

Role: Afraz and Tahseen *Priority:* Medium

4.3.4. Milestone: Compare physical hardware data with simulation data

Cross-reference the latency and API load metrics from the Pixel phones against both Kind and KWOK baselines to prove the validity of our simulation analysis.

Role: Felicia and Blake *Priority:* High

4.3.5. Milestone: Test on homogeneous vs heterogeneous mixture of hardware

Mix Pixel phone worker nodes with other phones or boards to evaluate how Karmada federates workloads across devices with different compute capabilities.

Role: Afraz and Tahseen *Priority:* Low

4.3.6. Milestone: Finalize decision making rules for when one cluster should become many

Gather all data from Kind, KWOK, and physical hardware to produce our final, documented data-driven scaling rules regarding cluster of clusters federation.

Role: All *Priority:* High

4.4. Deliverable 6: Final Delivery/Presentation - Week 10/11**4.4.1. Milestone: Clean up GitHub repository and ensure proper documentation**

Organize all scripts, YAML configurations, and README instructions so future teams can seamlessly recreate both our simulation and hardware environments.

Role: All *Priority:* High

4.4.2. Milestone: Complete project webpresence detailing architecture and how to reproduce results

Publish our environment/dependencies, workflow/process, Grafana dashboards, discoveries, scaling rules, and future possible work to our project's GitHub page.

Role: All *Priority:* High

4.4.3. Milestone: Complete final video, report, and presentation

Compile our quarter-long findings into a final video, report, and presentation.

Role: All *Priority:* High